

```

/**
 * Payload Decoder for Datacake
 * Based on Milesight UC51x v3 payload format
 * Copyright 2023 Milesight IoT
 *
 * @product UC51x v3
 * Sample Payload:
03010004c800000000005010006c800000000070100080100
 */
function Decoder(bytes, port) {
    return milesight(bytes);
}

function milesight(bytes) {
    var decoded = {};

    for (var i = 0; i < bytes.length; ) {

        // Guard: need at least 2 bytes for channel_id + channel_type
        if (i + 1 >= bytes.length) break;

        var channel_id = bytes[i++];
        var channel_type = bytes[i++];

        // BATTERY
        if (channel_id === 0x01 && channel_type === 0x75) {
            if (i + 1 > bytes.length) break;
            decoded.battery = bytes[i];
            i += 1;
        }
        // VALVE 1 status
        else if (channel_id === 0x03 && channel_type === 0x01) {
            if (i + 1 > bytes.length) break;
            decoded.valve1 = bytes[i] === 0 ? "off" : "on"; // ← human-readable
string
            i += 1;
        }
        // VALVE 2 status
        else if (channel_id === 0x05 && channel_type === 0x01) {
            if (i + 1 > bytes.length) break;
            decoded.valve2 = bytes[i] === 0 ? "off" : "on"; // ← human-readable
string
            i += 1;
        }
        // VALVE 1 Pulse counter
        else if (channel_id === 0x04 && channel_type === 0xc8) {
            if (i + 4 > bytes.length) break;

```

```

        decoded.valve1_pulse = readUInt32LE(bytes.slice(i, i + 4));
        i += 4;
    }
    // VALVE 2 Pulse counter
    else if (channel_id === 0x06 && channel_type === 0xc8) {
        if (i + 4 > bytes.length) break;
        decoded.valve2_pulse = readUInt32LE(bytes.slice(i, i + 4));
        i += 4;
    }
    // GPIO 1
    else if (channel_id === 0x07 && channel_type === 0x01) {
        if (i + 1 > bytes.length) break;
        decoded.gpio_1 = bytes[i] === 0 ? "off" : "on"; // ← human-readable
string
        i += 1;
    }
    // GPIO 2
    else if (channel_id === 0x08 && channel_type === 0x01) {
        if (i + 1 > bytes.length) break;
        decoded.gpio_2 = bytes[i] === 0 ? "off" : "on"; // ← human-readable
string
        i += 1;
    }
    // HISTORY
    else if (channel_id === 0x20 && channel_type === 0xce) {
        if (i + 9 > bytes.length) break;
        if (!decoded.history) decoded.history = [];

        var timestamp = readUInt32LE(bytes.slice(i, i + 4));
        var data      = bytes[i + 4];
        var status    = (data & 0x01) === 0 ? "off" : "on";
        var mode      = ((data >> 1) & 0x01) === 0 ? "counter" : "gpio";
        var gpio      = ((data >> 2) & 0x01) === 0 ? "off" : "on";
        var index     = ((data >> 4) & 0x01) === 0 ? "1" : "2";
        var pulse     = readUInt32LE(bytes.slice(i + 5, i + 9));

        var payload = { mode: mode, timestamp: timestamp };

        if (mode === "gpio") {
            payload["valve_" + index] = status;
            payload["gpio_" + index] = gpio;
        } else {
            payload["valve_" + index] = status;
            payload["valve_" + index + "_pulse"] = pulse;
        }

        decoded.history.push(payload);

```

```

        i += 9;
    }
    // Unknown channel — skip gracefully instead of hard break
    else {
        i++; // ← was: break — silently skips unknown TLV instead of discarding
rest of payload
    }
}

// Datacake: flatten history array into indexed fields (arrays are not supported
as field values)
if (decoded.history && decoded.history.length > 0) {
    decoded.history.forEach(function(entry, idx) {
        var prefix = "history_" + (idx + 1) + "_";
        Object.keys(entry).forEach(function(key) {
            decoded[prefix + key] = entry[key];
        });
    });
    delete decoded.history; // ← remove the raw array; Datacake cannot store
it as a field
}

// LoRa radio metadata (Datacake injects normalizedPayload globally)
try {
    var gw = (normalizedPayload.gateways &&
normalizedPayload.gateways[0]) || {};
    decoded.lora_rssi    = gw.rssi || 0;
    decoded.lora_snr     = gw.snr  || 0;
    decoded.lora_datarate = normalizedPayload.data_rate || "not retrievable";
} catch (e) {
    decoded.lora_rssi    = 0;
    decoded.lora_snr     = 0;
    decoded.lora_datarate = "not retrievable";
}

// Datacake expects an array of { field, value } objects
return Object.keys(decoded).map(function(key) {
    return { field: key.toUpperCase(), value: decoded[key] };
});
}

/* *****
* bytes to number helpers
***** */
function readUInt32LE(bytes) {
    var value = (bytes[3] << 24) + (bytes[2] << 16) + (bytes[1] << 8) + bytes[0];
    return (value & 0xffffffff) >>> 0;
}

```

```
}
```

```
function readInt32LE(bytes) {  
  var ref = readUInt32LE(bytes);  
  return ref > 0x7fffffff ? ref - 0x100000000 : ref;  
}
```